

Experiment No. :- 06

1] Make A Folder on Local Machine

2] Access that folder on VSCode

Example:

ai-docker-app(Folder name is user defined)

Open this folder in VS Code (or any editor).

3] Create app.py (Make a file in VsCode name with extension .py)

eg.

```
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return "AI Model is Running!"
```

```
@app.route('/predict', methods=['POST'])
def predict():
    data = request.json['number']
    result = data * 2
    return jsonify({'result': result})
```

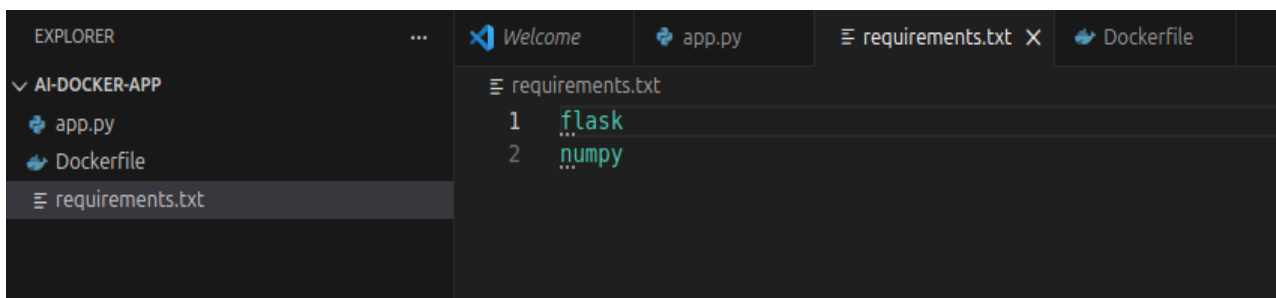
```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

4] Create requirement.txt (inside the VSCode in same folder)

add this info

flask

numpy



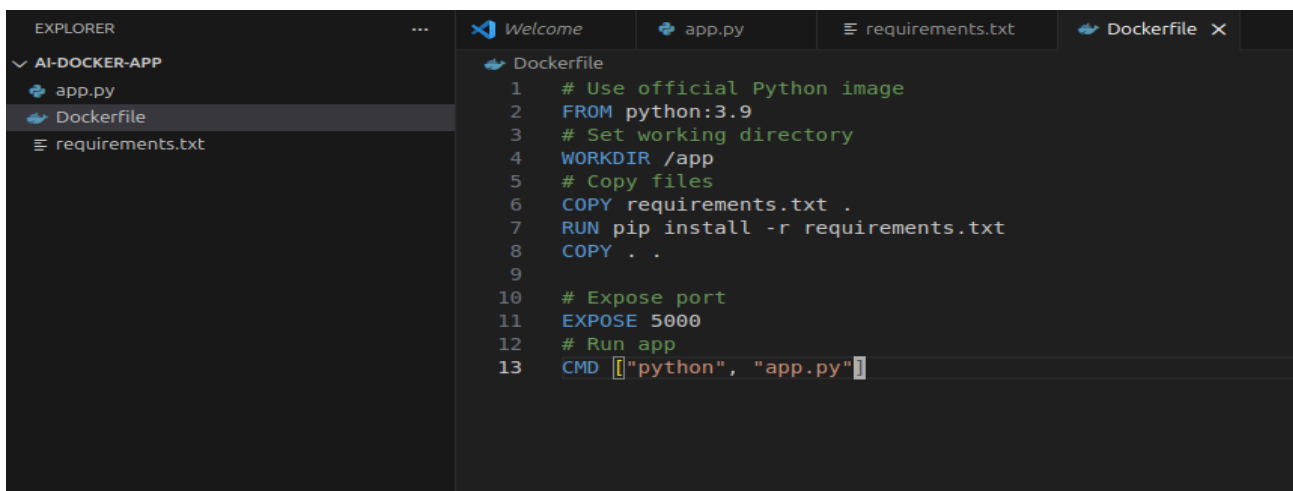
5] Create Dockerfile (inside the VSCode in same folder)

add this info in Dockerfile

```
# Use official Python image
FROM python:3.9
# Set working directory
WORKDIR /app
# Copy files
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

```
# Expose port
EXPOSE 5000
# Run app
CMD ["python", "app.py"]
```

eg :-

A screenshot of the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'AI-DOCKER-APP' with files 'app.py', 'Dockerfile', and 'requirements.txt'. The 'Dockerfile' is selected. The main editor area shows the content of the Dockerfile, which is a 13-line script for building a Python application container. The code is color-coded: comments are green, Dockerfile instructions are blue, and file names or paths are orange. The script starts with 'FROM python:3.9', sets the working directory to '/app', copies 'requirements.txt' and the current directory, installs dependencies, exposes port 5000, and runs 'python app.py'.

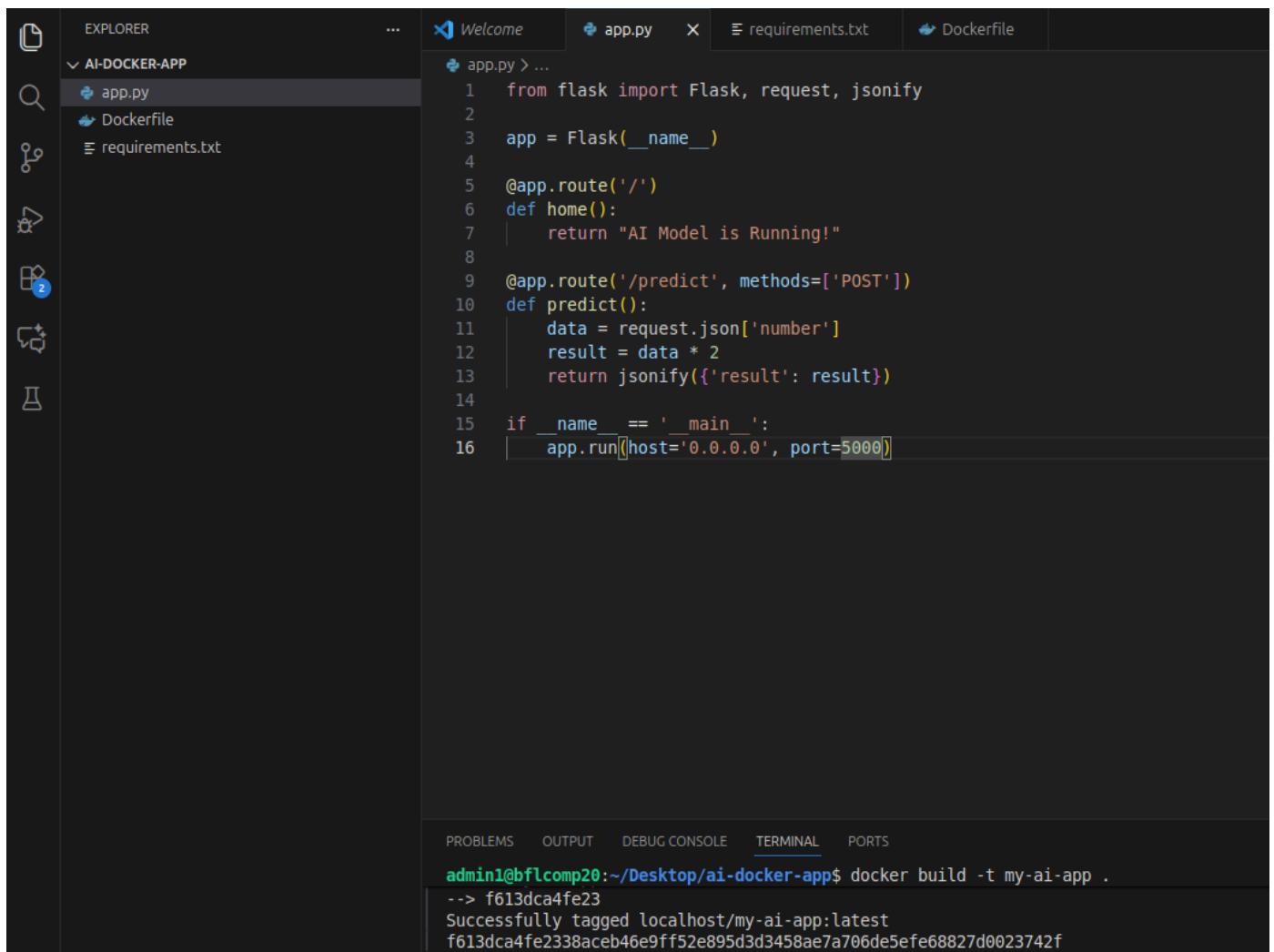
6] Build Docker Image

Open terminal inside your project folder:

Run:

```
docker build -t my-ai-app .
```

eg.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project named 'AI-DOCKER-APP' with files 'app.py', 'Dockerfile', and 'requirements.txt'. The main editor displays the 'app.py' file, which contains a Flask application. The application has a home route and a predict route that takes a JSON input and returns a doubled value. The terminal at the bottom shows the command 'docker build -t my-ai-app .' being executed, resulting in a successful build and tagging of the image as 'localhost/my-ai-app:latest'.

```
1 from flask import Flask, request, jsonify
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return "AI Model is Running!"
8
9 @app.route('/predict', methods=['POST'])
10 def predict():
11     data = request.json['number']
12     result = data * 2
13     return jsonify({'result': result})
14
15 if __name__ == '__main__':
16     app.run(host='0.0.0.0', port=5000)
```

```
admin1@bflcomp20:~/Desktop/ai-docker-app$ docker build -t my-ai-app .
--> f613dca4fe23
Successfully tagged localhost/my-ai-app:latest
f613dca4fe2338aceb46e9ff52e895d3d3458ae7a706de5efe68827d0023742f
```

7] Run Docker Container

Run this:

```
docker run -p 5000:5000 my-ai-app
```

eg .

The screenshot shows a VS Code editor with a project named 'AI-DOCKER-APP'. The Explorer sidebar on the left lists the files: 'app.py', 'Dockerfile', and 'requirements.txt'. The main editor window displays the code for 'app.py', which is a Flask application. The code includes imports for Flask, request, and jsonify, a Flask app instance, a home route, a predict route, and a main block to run the app on port 5000.

```
1 from flask import Flask, request, jsonify
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return "AI Model is Running!"
8
9 @app.route('/predict', methods=['POST'])
10 def predict():
11     data = request.json['number']
12     result = data * 2
13     return jsonify({'result': result})
14
15 if __name__ == '__main__':
16     app.run(host='0.0.0.0', port=5000)
```

Below the code editor, the TERMINAL panel shows the execution of Docker commands. The first command is 'docker build -t my-ai-app .', which successfully builds the image and tags it as 'localhost/my-ai-app:latest'. The second command is 'docker run -p 5000:5000 my-ai-app', which starts the container. The terminal output shows the container's logs, including a warning that this is a development server and not for production use, and the Flask app serving requests on port 5000.

```
admin1@bflcomp20:~/Desktop/ai-docker-app$ docker build -t my-ai-app .
--> f613dca4fe23
Successfully tagged localhost/my-ai-app:latest
f613dca4fe238aceb46e9ff52e895d3d3458ae7a706de5efe68827d0023742f
admin1@bflcomp20:~/Desktop/ai-docker-app$ docker run -p 5000:5000 my-ai-app
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.100:5000
Press CTRL+C to quit
10.0.2.100 - - [25/Feb/2026 05:09:01] "GET / HTTP/1.1" 200 -
10.0.2.100 - - [25/Feb/2026 05:09:03] "GET / HTTP/1.1" 200 -
10.0.2.100 - - [25/Feb/2026 05:10:15] "GET / HTTP/1.1" 200 -
10.0.2.100 - - [25/Feb/2026 05:10:16] "GET / HTTP/1.1" 200 -
```

8] Test It On Local Machine

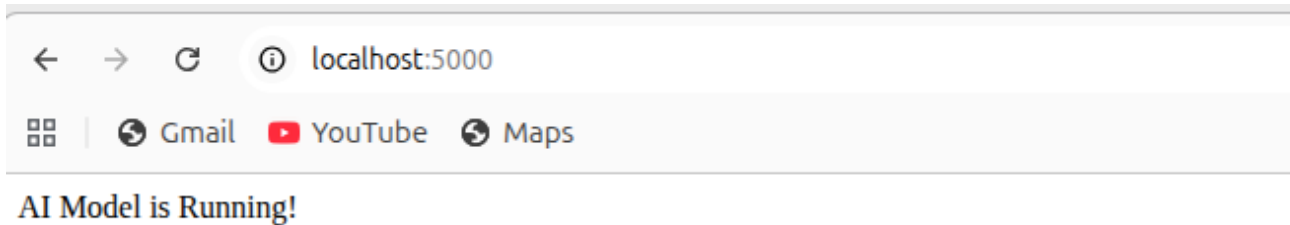
Open browser and go to:

<http://localhost:5000>

You should see:

AI Model is Running!

Eg .



9] Test Prediction API

Open another terminal and run:

```
curl -X POST http://localhost:5000/predict \  
-H "Content-Type: application/json" \  
-d '{"number": 5}'
```

You should see: (OUTPUT)

```
{"result":10}
```

